Descriptive Analytics

Descriptive analytics is the process of summarising data to gain insights into the data set. It is used to uncover patterns and trends in data, enabling organisations to make data-driven decisions. Descriptive analytics can be used to identify customer preferences, improve customer experience, and identify potential areas of improvement or opportunity.

Import/install necessary packages

```
import pandas as pd
```

```
data = {'BL': [5.6,5,5.8,3,7,4.6,7,5.6,5,5.8],
        'BAS': [6,7,5.25,5.5,6.5,4.75,6,5.5,6.25,7],
        'BAW': [4,5,6,2,7,4.25,6,5,5.25,6],
        'PQ': [6,7,6,5.25,5.25,6,1,5.25,6,6.25],
        'OBE': [6,5,6,5.5,7,6,5,6,7,6.5]}
df = pd.DataFrame(data)
```

Summarise Data

Get the meta data information

Meta data is a set of data that describes other data, providing contextual information about the data that is being analysed.

```
df.info()
```

```
<class 'pandas.core.frame.DataFrame'>
RangeIndex: 10 entries, 0 to 9
Data columns (total 5 columns):
 #   Column  Non-Null Count  Dtype
---  ------  --------------  -----
 0   BL      10 non-null     float64
 1   BAS     10 non-null     float64
 2   BAW     10 non-null     float64
 3   PQ      10 non-null     float64
 4   OBE     10 non-null     float64
dtypes: float64(5)
memory usage: 528.0 bytes
```

Print

1. Number of records

2. Number of rows and columns
3. Describe index
4. Describe the DataFrame columns
5. The info on DataFrame
6. Number of non-NA values

```
print(len(df),"\n")
print(df.shape,"\n")
print(df.index,"\n")
print(df.columns,"\n")
print(df.info(),"\n")
print(df.count(),"\n")
```

```
10

(10, 5)

RangeIndex(start=0, stop=10, step=1)

Index(['BL', 'BAS', 'BAW', 'PQ', 'OBE'], dtype='object')

<class 'pandas.core.frame.DataFrame'>
RangeIndex: 10 entries, 0 to 9
Data columns (total 5 columns):
 #   Column  Non-Null Count  Dtype
---  ------  --------------  -----
 0   BL      10 non-null     float64
 1   BAS     10 non-null     float64
 2   BAW     10 non-null     float64
 3   PQ      10 non-null     float64
 4   OBE     10 non-null     float64
dtypes: float64(5)
memory usage: 528.0 bytes
None

BL     10
BAS    10
BAW    10
PQ     10
OBE    10
dtype: int64
```

Quickly view the first 5 rows of the data set

```
df.head(5)
```

	BL	BAS	BAW	PQ	OBE
0	5.6	6.00	4.0	6.00	6.0
1	5.0	7.00	5.0	7.00	5.0
2	5.8	5.25	6.0	6.00	6.0
3	3.0	5.50	2.0	5.25	5.5
4	7.0	6.50	7.0	5.25	7.0

Summary Statistics

Summary statistics are numerical summaries of data that provide a concise overview of the characteristics of a dataset. These summaries can include the mean, median, mode, range, and standard deviation.

```
df.describe
```

```
<bound method NDFrame.describe of     BL   BAS   BAW    PQ  OBE
0  5.6  6.00  4.00  6.00  6.0
1  5.0  7.00  5.00  7.00  5.0
2  5.8  5.25  6.00  6.00  6.0
3  3.0  5.50  2.00  5.25  5.5
4  7.0  6.50  7.00  5.25  7.0
5  4.6  4.75  4.25  6.00  6.0
6  7.0  6.00  6.00  1.00  5.0
7  5.6  5.50  5.00  5.25  6.0
8  5.0  6.25  5.25  6.00  7.0
9  5.8  7.00  6.00  6.25  6.5>
```

```
df.describe()
```

	BL	BAS	BAW	PQ	OBE
count	10.000000	10.000000	10.000000	10.000000	10.000000
mean	5.440000	5.975000	5.050000	5.400000	6.000000
std	1.165428	0.740214	1.398412	1.638088	0.707107
min	3.000000	4.750000	2.000000	1.000000	5.000000
25%	5.000000	5.500000	4.437500	5.250000	5.625000
50%	5.600000	6.000000	5.125000	6.000000	6.000000
75%	5.800000	6.437500	6.000000	6.000000	6.375000
max	7.000000	7.000000	7.000000	7.000000	7.000000

```
df.describe().transpose()
```

	count	mean	std	min	25%	50%	75%	max
BL	10.0	5.440	1.165428	3.00	5.0000	5.600	5.8000	7.0
BAS	10.0	5.975	0.740214	4.75	5.5000	6.000	6.4375	7.0
BAW	10.0	5.050	1.398412	2.00	4.4375	5.125	6.0000	7.0
PQ	10.0	5.400	1.638088	1.00	5.2500	6.000	6.0000	7.0
OBE	10.0	6.000	0.707107	5.00	5.6250	6.000	6.3750	7.0

Get the average

```
df.mean()
```

```
BL     5.440
BAS    5.975
BAW    5.050
PQ     5.400
OBE    6.000
dtype: float64
```

Get the summary for Brand Loyalty

```
df.describe().BL
```

```
count    10.000000
mean      5.440000
std       1.165428
min       3.000000
25%       5.000000
50%       5.600000
75%       5.800000
max       7.000000
Name: BL, dtype: float64
```

Skewness and Kurtosis

Skewness is a measure of the asymmetry of a data distribution, while kurtosis is a measure of the peakedness of a data distribution. Both measures can help identify underlying patterns in data that can indicate potential outliers or errors in data collection.

```
df.skew()
```

```
BL    -0.666452
BAS   -0.039168
BAW   -1.018175
PQ    -2.515809
OBE    0.000000
dtype: float64
```

```
df.kurtosis()
```

```
BL      1.327503
BAS    -0.762976
BAW     1.657009
PQ      7.252133
OBE    -0.738095
dtype: float64
```

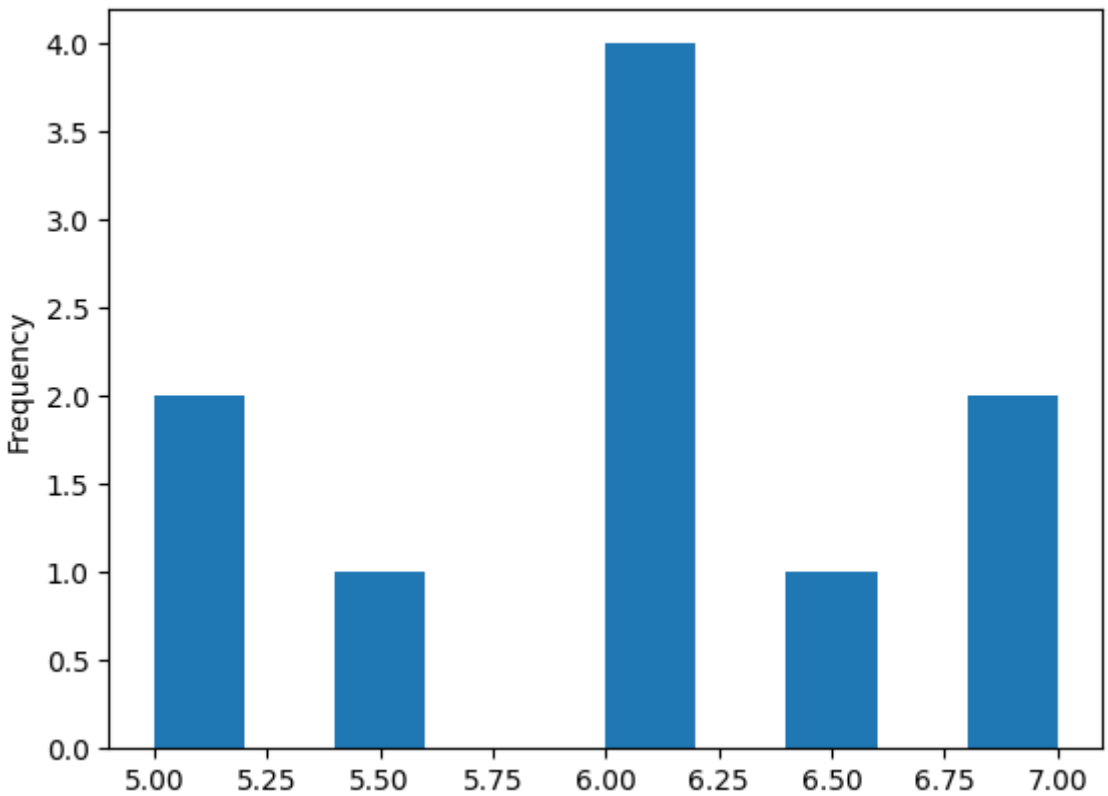
Chart visualisation

Display Histogram - Overall Brand Equity

A histogram is a graphical representation of data that is used to show the frequency distribution of a given data set. It is a bar chart that groups a range of data into different bins and displays the number of data points that fall within each bin.

```
df.OBE.plot.hist()
```

```
<Axes: ylabel='Frequency'>
```

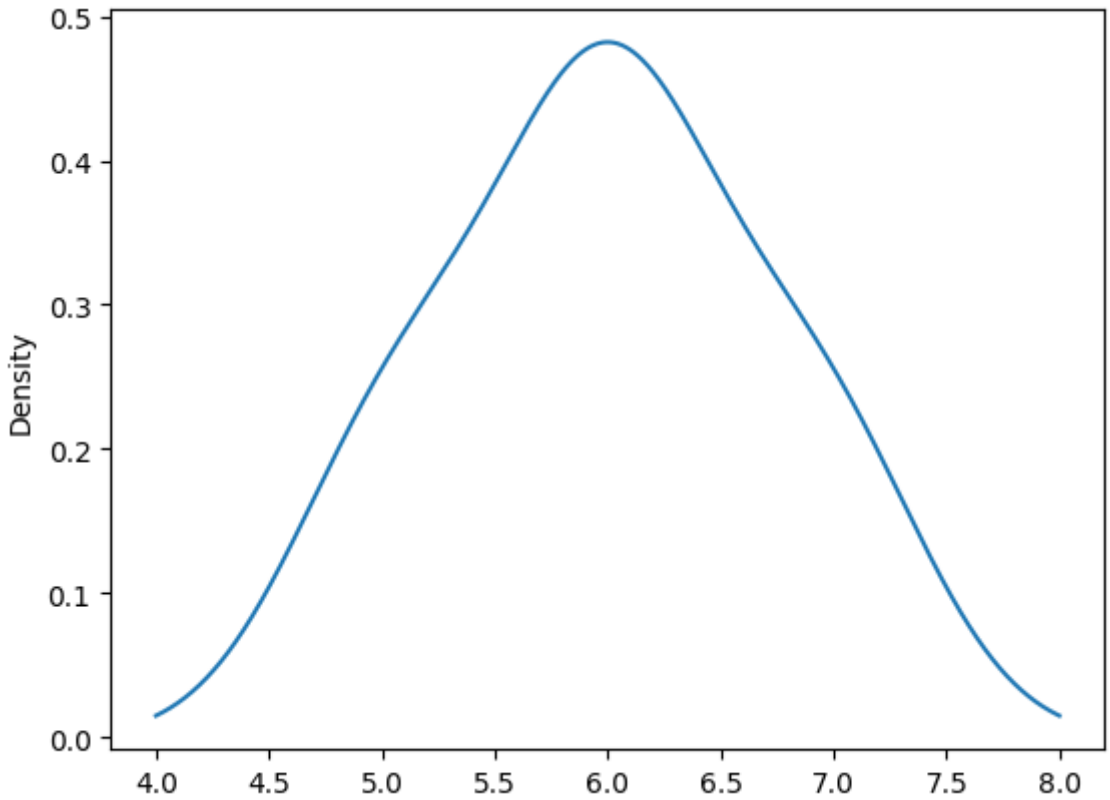


Display a Density Plot

KDE Density Plot: A KDE density plot is a graphical representation of data that displays the probability density of a given data set. It is a smooth curve that shows the estimated probability density of each data point.

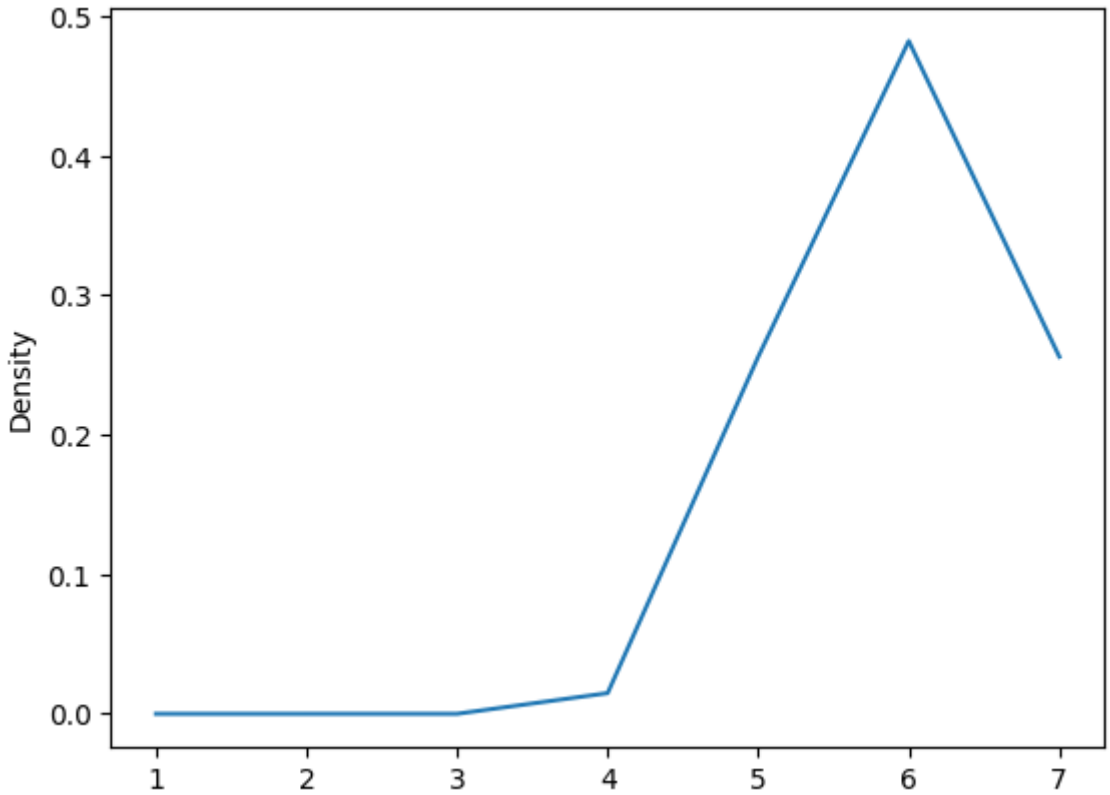
```
df.OBE.plot.kde()
```

```
<Axes: ylabel='Density'>
```




```
df.OBE.plot.kde(ind=[1, 2, 3, 4, 5, 6, 7])
```

```
<Axes: ylabel='Density'>
```

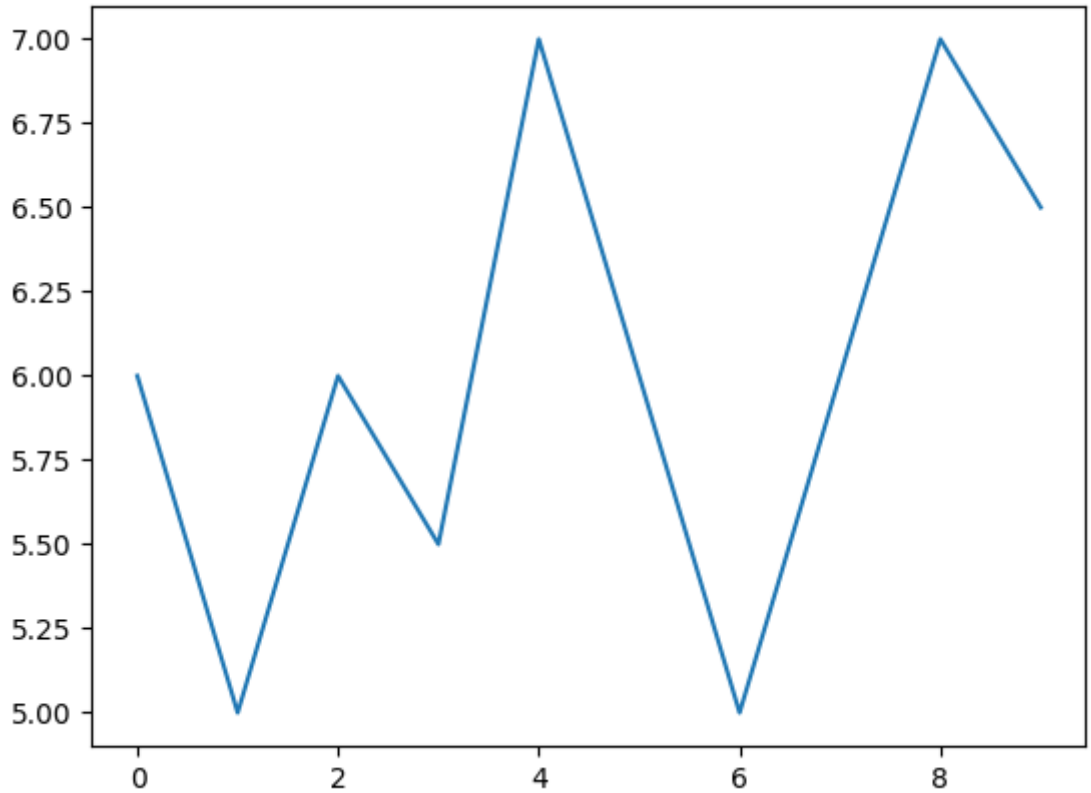


Display a Line Chart - OBE

A line plot is a graph that shows a set of data points connected by straight lines. It is used to observe trends and patterns in the data, and can be used to compare multiple data sets on the same graph.

```
df.OBE.plot()
```

```
<Axes: >
```

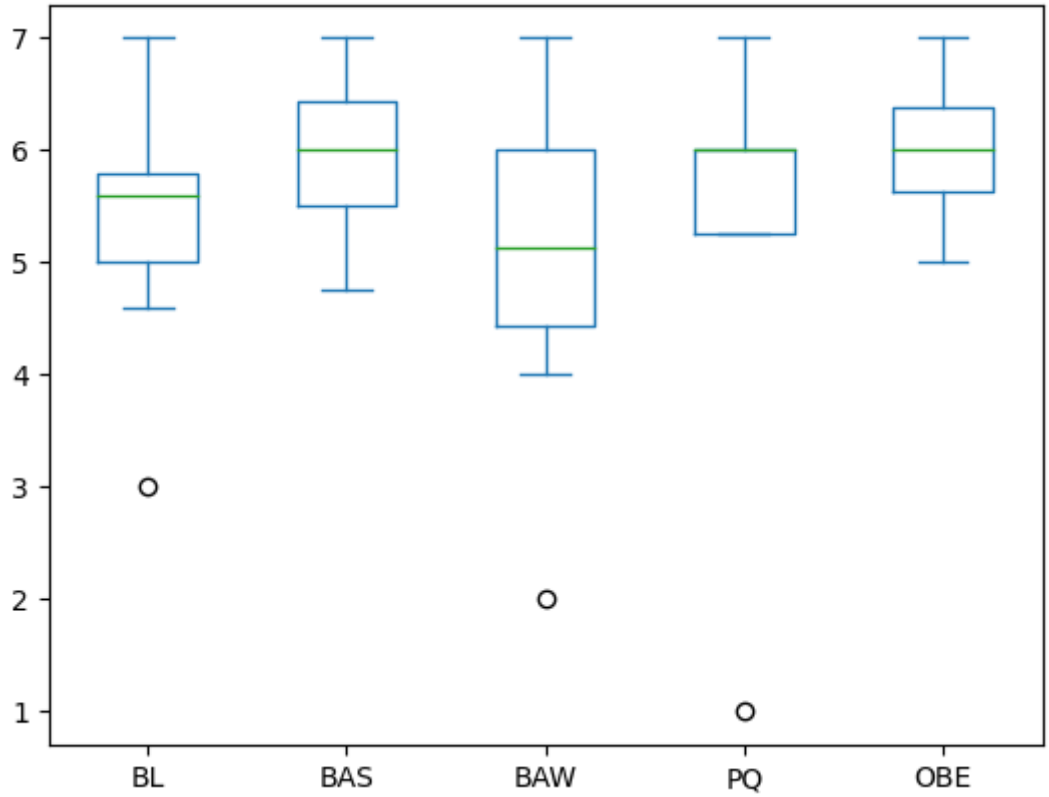


Display a Box Plot - OBE

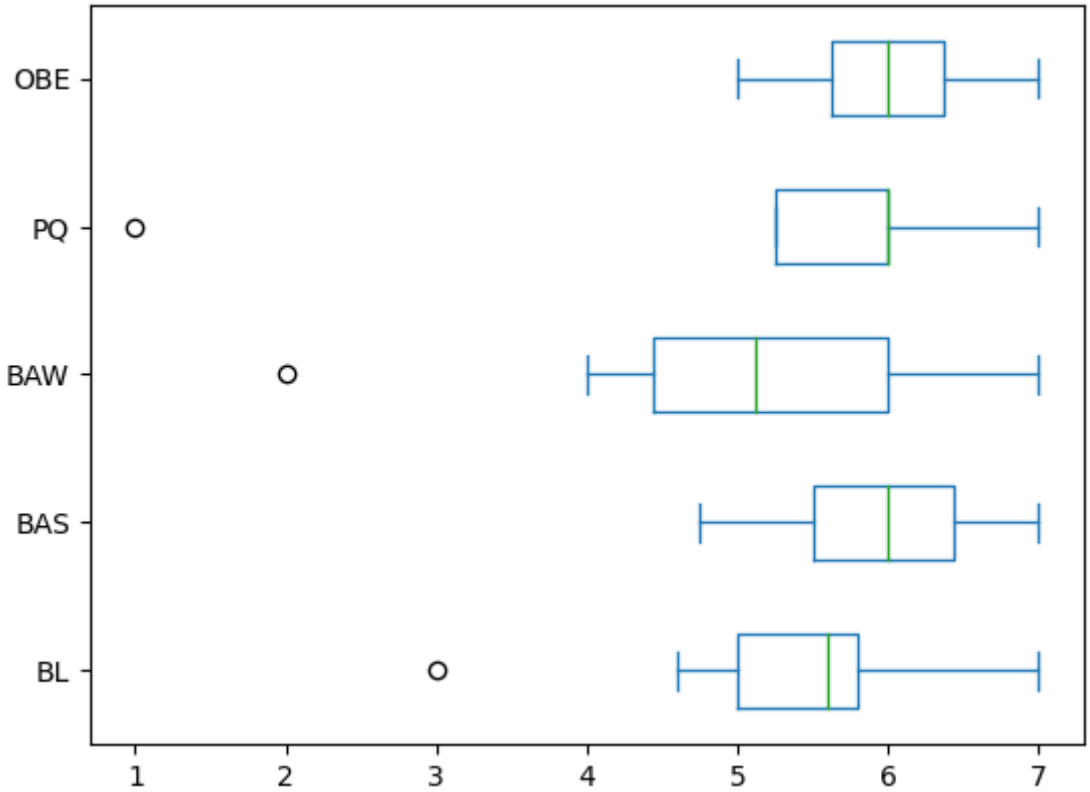
A box plot is a graphical representation of data that shows the median, quartiles, maximum, and minimum values in a dataset. It is used to compare multiple datasets, identify outliers, and assess overall data distribution and spread.

```
df.plot.box()
```

```
<Axes: >
```

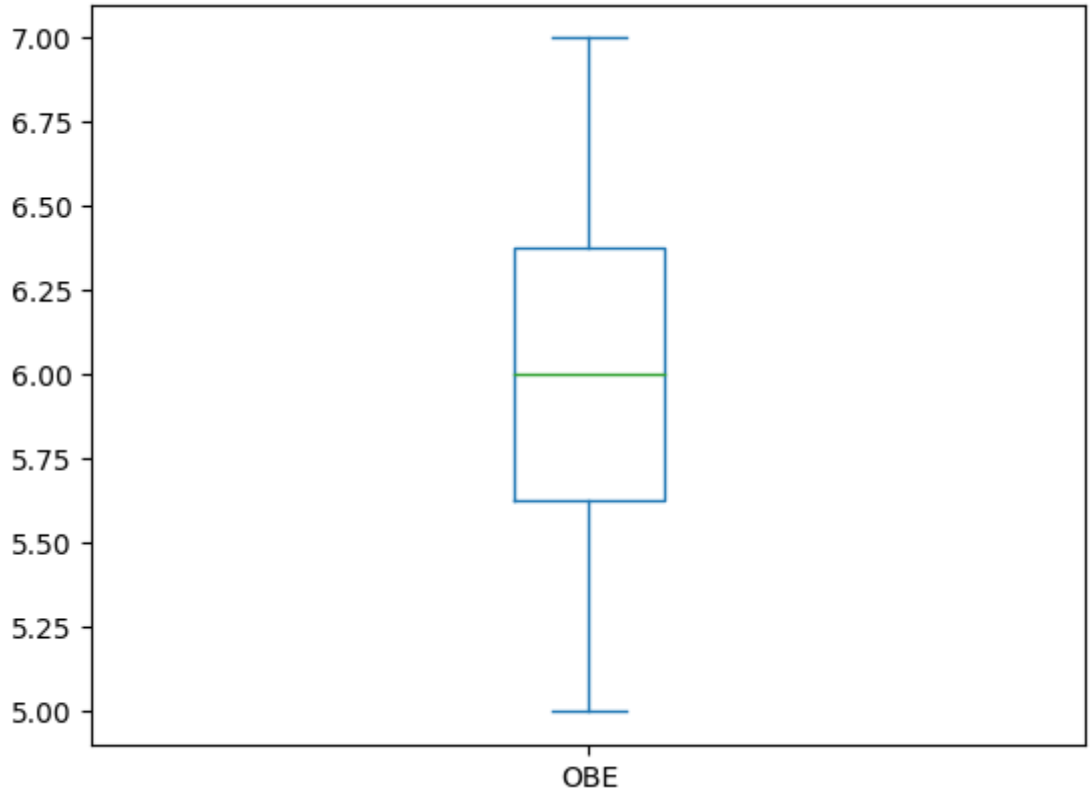



```
df.plot.box(vert=False);
```




```
df.OBE.plot.box()
```

```
<Axes: >
```

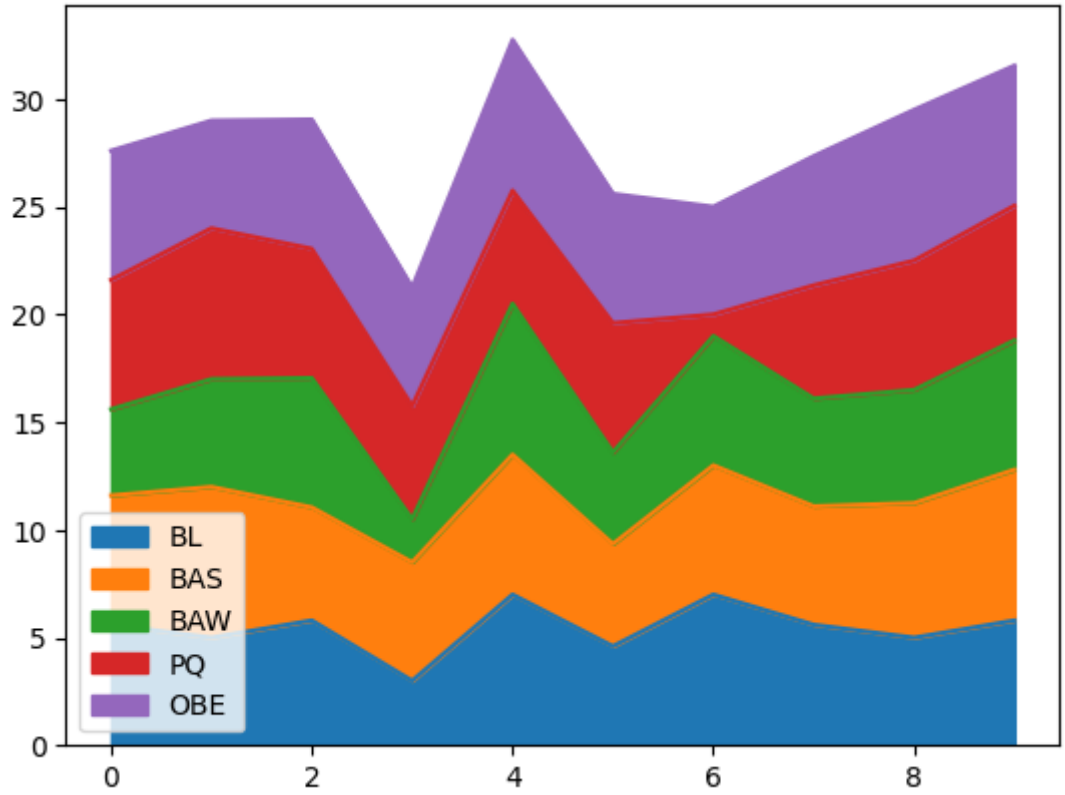


Display an Area Plot

An area plot is a two-dimensional graph that displays quantitative data through a filled polygon, which is created by connecting a sequence of points. It is used to visualise the changes in data over a certain period of time, track cumulative totals, and compare the relationship between two or more variables.

```
df.plot.area()
```

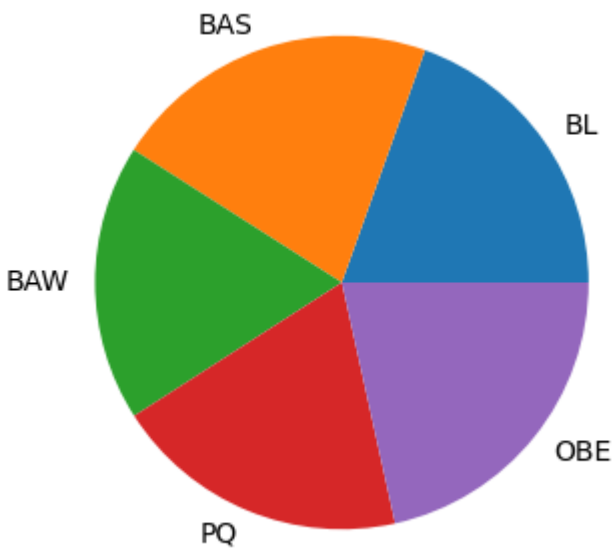
```
<Axes: >
```



Display a Pie Plot

Pie plots are two-dimensional graphs that display data as a circle divided into sections, with each section representing a proportion of the whole. Pie plots are used to compare the relative size of different categories or groups within a dataset.

```
series = df.sum()
series.plot.pie(figsize=(4, 4));
```

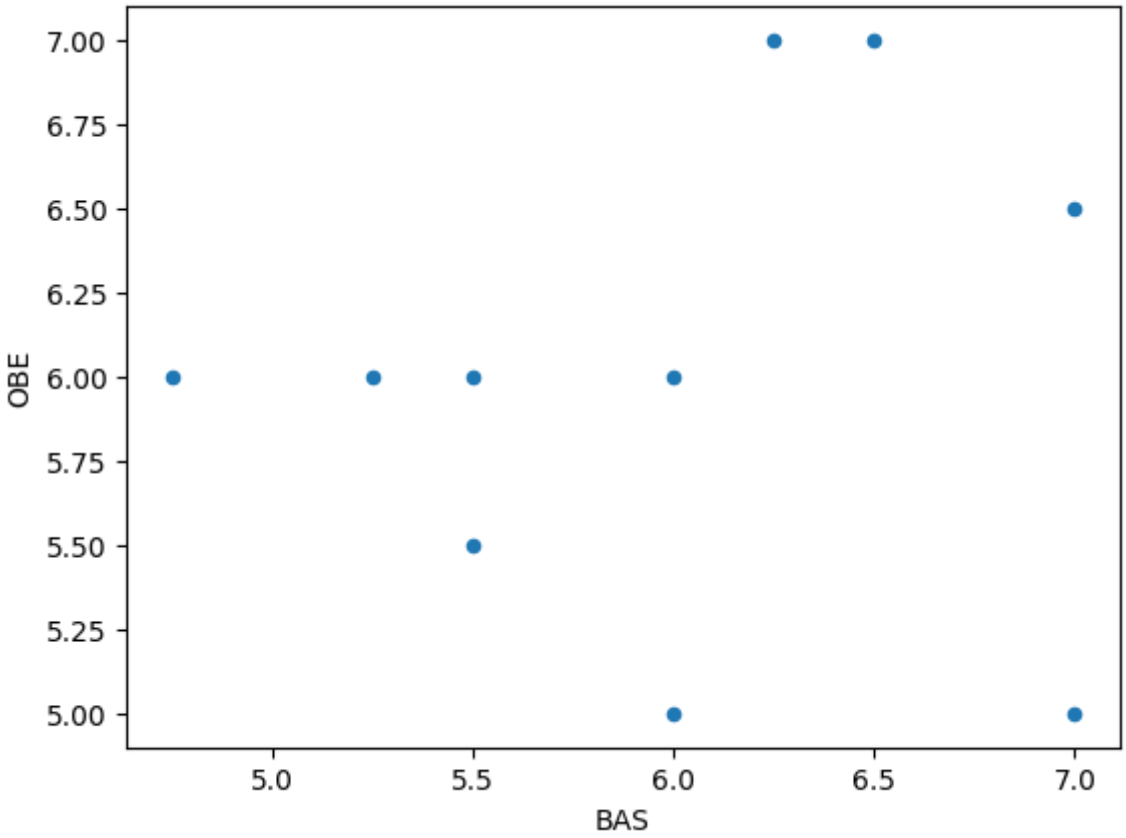


Display a Scatter Plot Using Pairs of Variables (Brand Association vs Overall Brand Equity)

Scatter plots are two-dimensional graphs that display data as a collection of points, with each point representing the values of two variables. The position of the point on the graph indicates how the two variables are related, allowing for easy identification of trends and patterns.

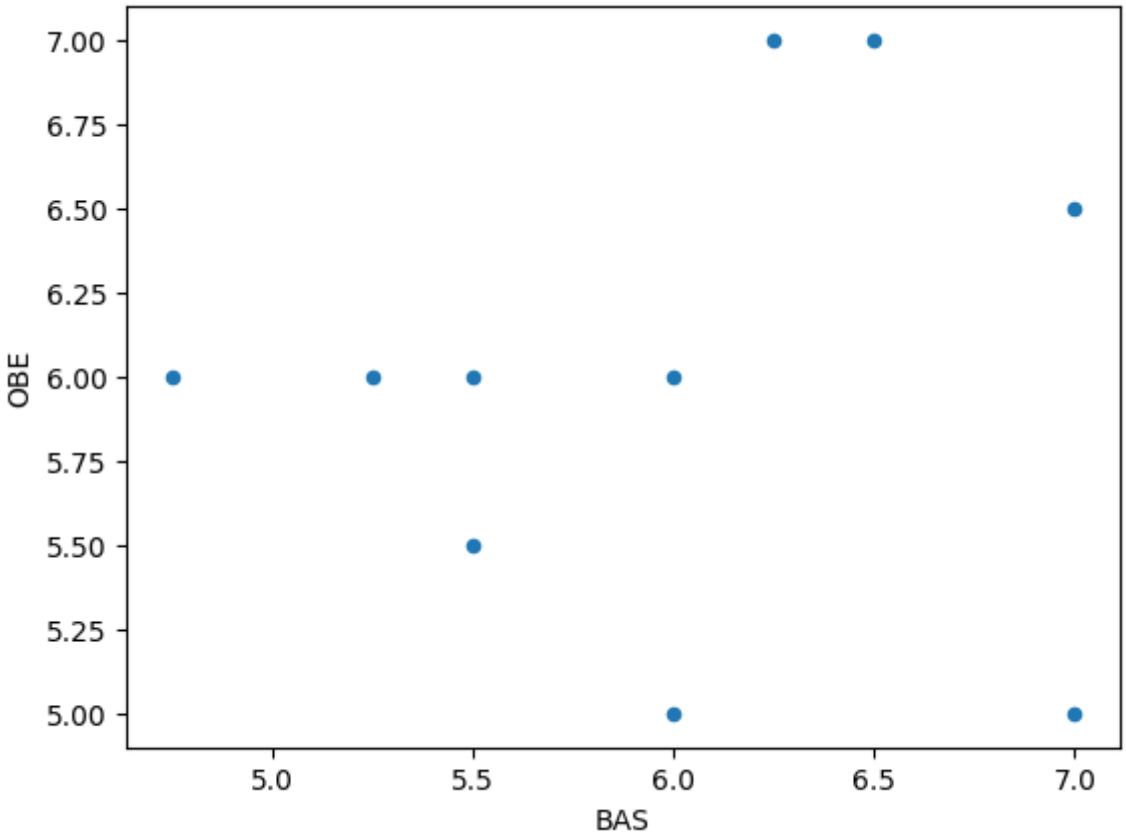
```
df.plot.scatter(x='BAS',y='OBE')
```

```
<Axes: xlabel='BAS', ylabel='OBE'>
```




```
df.plot(x='BAS',y='OBE',kind="scatter")
```

```
<Axes: xlabel='BAS', ylabel='OBE'>
```



Conclusion

Descriptive analytics is a powerful tool for uncovering patterns and trends in data. It can be used to identify customer preferences, improve customer experience, and identify potential areas of improvement or opportunity. However, it is limited in its ability to predict future outcomes or behaviour, and identify causal relationships between variables.

Exercise - 1

1. Display a scatter plot for Brand Loyalty and Brand Association.
2. Display a box plot for Perceived Quality.
3. Display a line plot for Brand Awareness.
4. Display a Kernel Density plot for Perceived Quality.
5. Display a histogram for Brand Loyalty.
6. Compute the skewness of Perceived Quality.
7. Compute the kurtosis of Brand Loyalty.
8. Print the summary statistics for Overall Brand Equity.
9. Print the bottom 5 rows of data.
10. Print the median for Brand Loyalty.

Exercise - 2

1. Download the data from Canvas BrandEquity_student_copy.xlsx
2. Upload the data to your Google Drive.
3. Use the following code to access the data from your Google Drive:

df = pd.read_excel ("/content/drive/MyDrive/Colab Notebooks/Data/BrandEquity_student_copy.xlsx") Note: use your own file path.

4. Rerun all the codes you have learned in this lab.

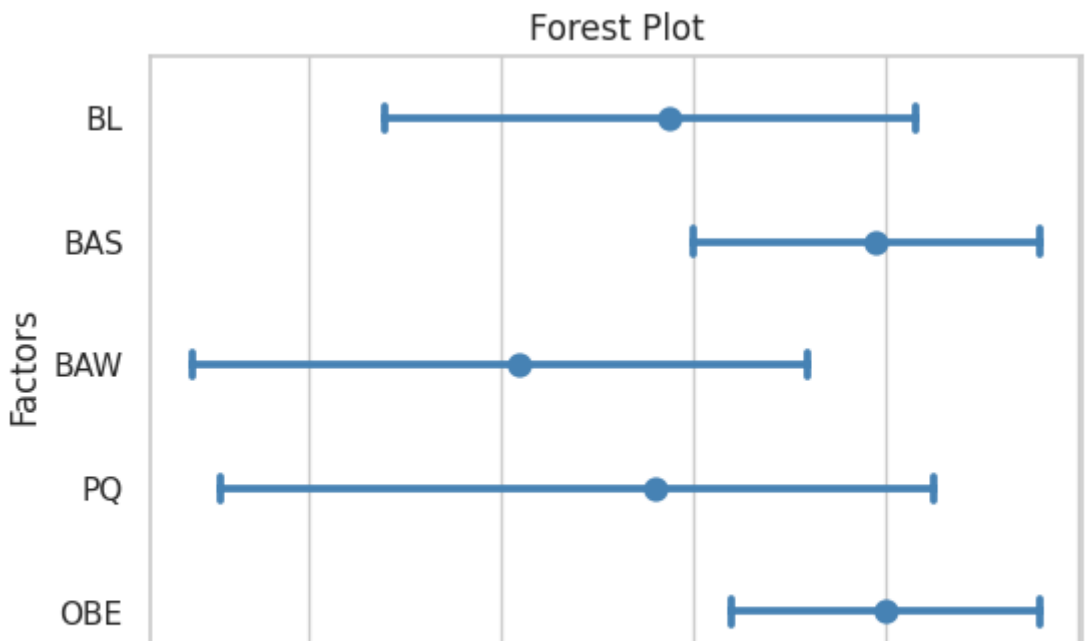
Extras:

Forest Plot

A forest plot is a graphical representation commonly used in meta-analysis and clinical research to display the point estimates and confidence intervals of multiple studies or data points for a particular set of variables. The plot resembles a "forest" with individual studies represented as horizontal lines (often with markers such as squares or circles) along a vertical axis.

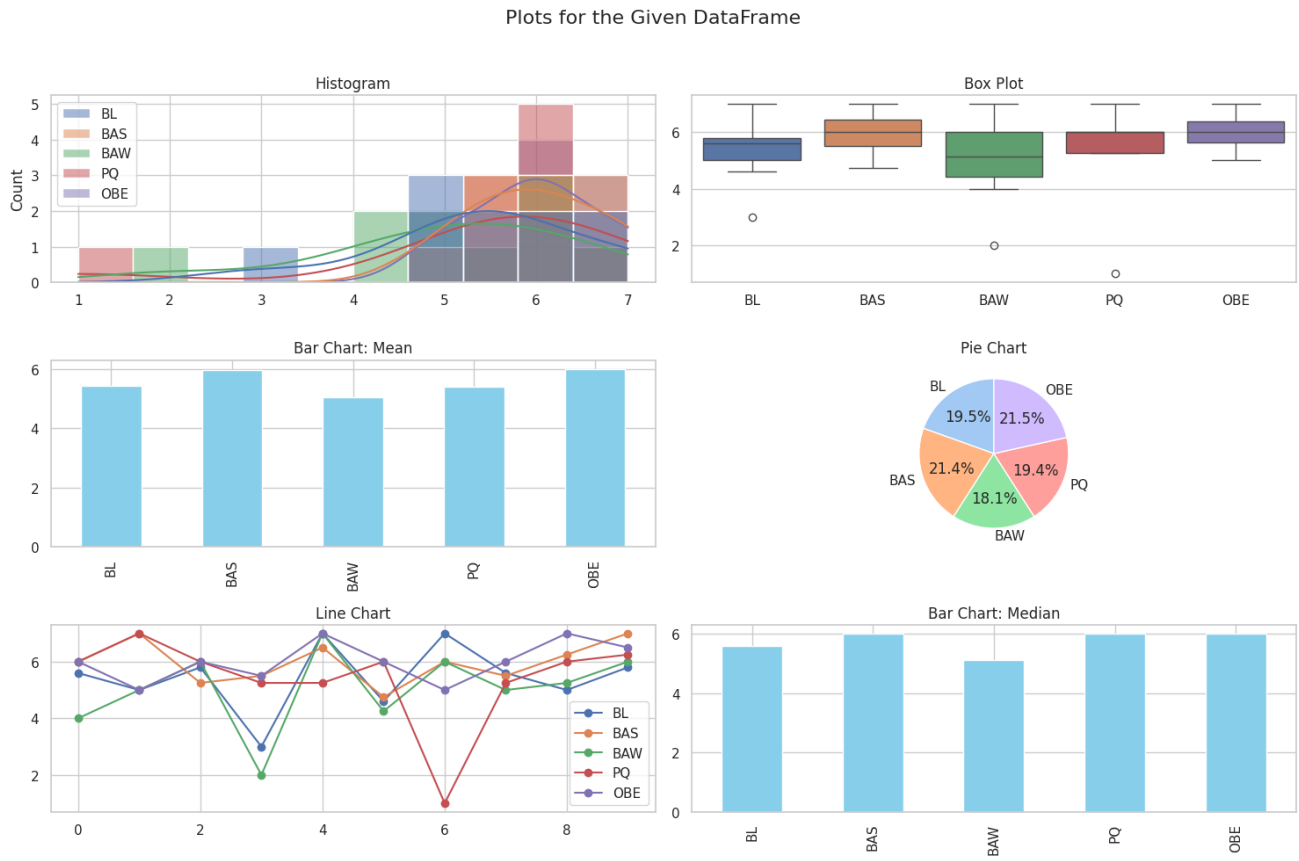
```
import matplotlib.pyplot as plt
import seaborn as sns

# Melt the DataFrame to long format for easy plotting
df_melted = pd.melt(df, var_name='Factors', value_name='Likert Scale')
print(df_melted) # Print it for your understanding

# Create a forest plot using seaborn
plt.figure(figsize=(6, 4))
sns.set(style="whitegrid")

# Plotting the forest plot (point plot)
sns.pointplot(x='Likert Scale', y='Factors', data=df_melted, color='steelblue', j

# Adding labels and title
plt.xlabel('Likert Scale')
plt.ylabel('Factors')
plt.title('Forest Plot')

plt.show()
```

```
13      BAS          5.50
14      BAS          6.50
15      BAS          4.75
16      BAS          6.00
17      BAS          5.50
18      BAS          6.25
19      BAS          7.00
20      BAW          4.00
21      BAW          5.00
22      BAW          6.00
23      BAW          2.00
24      BAW          7.00
25      BAW          4.25
26      BAW          6.00
27      BAW          5.00
28      BAW          5.25
29      BAW          6.00
30       PQ          6.00
31       PQ          7.00
32       PQ          6.00
33       PQ          5.25
34       PQ          5.25
35       PQ          6.00
36       PQ          1.00
37       PQ          5.25
38       PQ          6.00
39       PQ          6.25
40      OBE          6.00
41      OBE          5.00
42      OBE          6.00
43      OBE          5.50
44      OBE          7.00
45      OBE          6.00
46      OBE          5.00
47      OBE          6.00
48      OBE          7.00
49      OBE          6.50
<ipython-input-44-799df09388d6>:13: UserWarning:

The `join` parameter is deprecated and will be removed in v0.15.0. You can rer

  sns.pointplot(x='Likert Scale', y='Factors', data=df_melted, color='steelblu
```



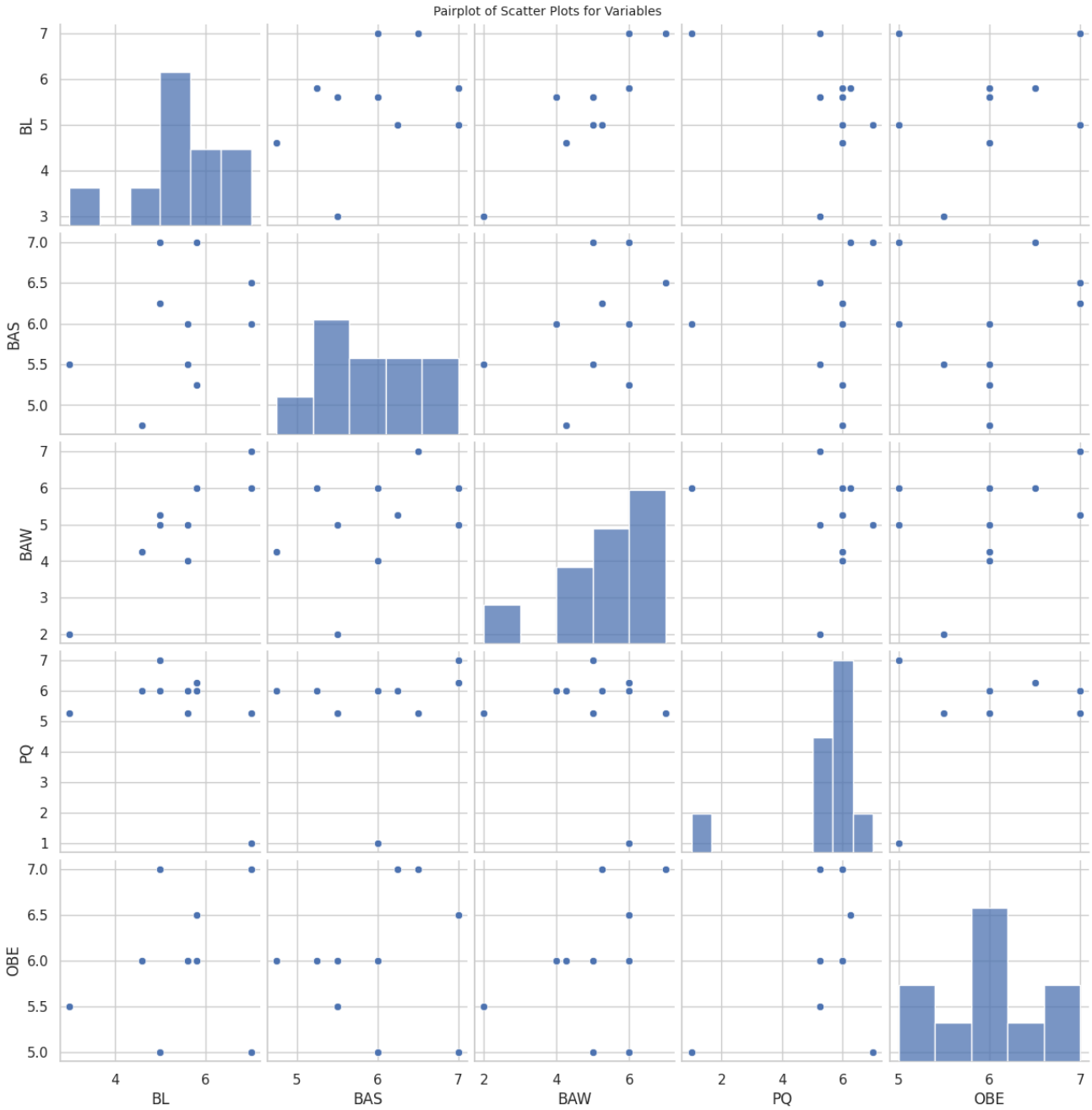
Plots

```
# Set up subplots for better organization
fig, axes = plt.subplots(nrows=3, ncols=2, figsize=(15, 10))
fig.suptitle('Plots for the Given DataFrame', fontsize=16)

# Histogram
sns.histplot(data=df, bins=10, kde=True, ax=axes[0, 0])
axes[0, 0].set_title('Histogram')

# Box Plot
sns.boxplot(data=df, ax=axes[0, 1])
axes[0, 1].set_title('Box Plot')

# Bar Chart
df.mean().plot(kind='bar', ax=axes[1, 0], color='skyblue')
axes[1, 0].set_title('Bar Chart: Mean')

# Pie Chart
df.sum().plot(kind='pie', autopct='%1.1f%%', ax=axes[1, 1], startangle=90, colors
axes[1, 1].set_title('Pie Chart')

# Line Chart
df.plot(kind='line', ax=axes[2, 0], marker='o')
axes[2, 0].set_title('Line Chart')

# Bar Chart (Median)
df.median().plot(kind='bar', ax=axes[2, 1], color='skyblue')
axes[2,1].set_title('Bar Chart: Median')

# Adjust layout for better spacing
plt.tight_layout(rect=[0, 0, 1, 0.96])
plt.show()
```



Pairplot of Scatter Plots

```
sns.pairplot(df)
plt.suptitle('Pairplot of Scatter Plots for Variables', y=1, fontsize=10)
plt.show()
```


Pairplot of Scatter Plots for Variables
BL
BAS
BAW
PQ
OBE